

Extended Abstract

Motivation Despite recent advances in reasoning capabilities, large language models (LLMs) continue to falter on arithmetic tasks requiring precise symbolic manipulation. In tasks like COUNTDOWN, a single numerical mistake results in a zero reward, presenting a brittle training environment under standard reinforcement learning. This sparsity severely limits the learning signal available to models during fine-tuning. We are motivated by the potential of *dense partial-credit rewards* to alleviate this bottleneck by assigning credit for partially correct outputs, thus guiding models more effectively toward reliable arithmetic behavior.

Implementation We built upon the Reinforce Leave-One-Out (RLOO) algorithm to fine-tune the Qwen-2.5-0.5B model on the COUNTDOWN dataset. As a warm-start, we first performed supervised fine-tuning (SFT) using a curated set of high-quality solutions to help the model learn basic formatting and response structure. We then introduced three partial-credit reward shaping mechanisms—Operand Multiset, Operand Longest Common Prefix (OLCP), and Standardized LCP (SLCP)—which evaluate response similarity to oracle solutions along increasingly structured dimensions. For each mechanism, we conducted extensive hyperparameter sweeps over rollout count (k), generation length, temperature, and reward scaling, using held-out evaluation to select stable and performant configurations.

Results Of the three mechanisms, OLCP yielded the best results across all evaluation metrics. It achieved the highest mean reward (0.3631), strict accuracy (30.8%), and format accuracy (85.9%), outperforming both baseline RLOO (29.3% strict accuracy) and multiset variants. Operand Multiset rewards, while dense, consistently underperformed, suggesting that frequent—but imprecise—feedback failed to meaningfully guide learning. SLCP, though promising in theory, was too computationally expensive due to recursive expression standardization and was omitted from full-scale experiments.

Discussion Our findings underscore that reward *precision* is as important as *density*. While Operand Multiset rewards made training signals more frequent, they lacked the structural granularity needed to push the model toward valid computation. This overabundance of imprecise credit likely led to reward hacking—where the model learns to exploit superficial patterns without improving underlying reasoning. In contrast, OLCP offered a more aligned and discriminative signal, rewarding outputs that preserved operand order and encouraging better-structured reasoning. Yet even with OLCP, training ultimately collapsed, revealing instability in RLOO learning dynamics. These results suggest that dense rewards must be *informative* and *targeted*, not merely frequent, and that symbolic RL tasks demand both nuanced feedback and stabilized exploration strategies.

Conclusion We demonstrate that partial-credit reward shaping, when carefully designed, can improve training dynamics and sample efficiency in arithmetic fine-tuning of LLMs. OLCP in particular enables better output structure and higher reward returns by providing precise, order-sensitive feedback. However, long-term training remains unstable due to reward collapse, indicating the need for future work in robust RL algorithms, better variance control, and structured intermediate supervision. Together, these insights move us closer to building language models capable of consistent and interpretable symbolic reasoning.

Partial Credit for Countdown

Alice Chen

Department of Computer Science
Stanford University
alicezyc@stanford.edu

Ryan Lian

Hasso Plattner Institute of Design
Stanford University
ryanlian@stanford.edu

Luke Tchang

Department of Computer Science
Stanford University
ltchang@stanford.edu

Abstract

Large language models (LLMs) remain unreliable at arithmetic reasoning, especially in symbolic tasks like COUNTDOWN where correctness is binary and rewards are sparse. We investigate whether dense partial-credit rewards can improve learning efficiency and output structure when fine-tuning small LLMs. Building on the Reinforce Leave-One-Out (RLOO) framework, we introduce reward mechanisms that assign credit based on operand overlap and prefix alignment with oracle solutions. Our experiments show that the Operand Longest Common Prefix (OLCP) reward improves mean return and format accuracy without sacrificing correctness, outperforming both the standard RLOO and multiset baselines. However, all methods suffer from eventual reward collapse, highlighting the need for better exploration and stability strategies. Our findings suggest that structured reward shaping is a promising direction for symbolic RL with language models.

1 Introduction

Despite remarkable advances in language understanding and reasoning (OpenAI et al., 2024; DeepSeek-AI et al., 2025), large language models (LLMs) continue to struggle with basic arithmetic tasks (Baeumel et al., 2025; Boye and Moell, 2025; Yan et al., 2025; Li et al., 2025). This limitation is particularly evident in benchmark tasks like COUNTDOWN (Xie, 2020), where a single numerical error leads to a total failure under binary correctness criteria. These brittle evaluation setups highlight the need for more informative learning signals during training.

One promising direction is the use of dense partial-credit rewards—intermediate feedback that assigns credit for being partially correct. In this work, we explore whether augmenting Reinforce Leave-One-Out (Ahmadian et al., 2024) (RLOO) fine-tuning with such dense signals can improve arithmetic capabilities in smaller-scale LLMs, specifically *Qwen-2.5-0.5B* (Qwen et al., 2025).

RLOO is a variant of the REINFORCE algorithm that reduces variance by holding one rollout out of the batch as a baseline. Despite performance improvements over SFT, we observe that RLOO still suffers from sparse rewards on particularly hard test cases where all k rollouts achieve a reward of 0 or 0.1.

To address this, we propose and evaluate several partial-credit reward mechanisms that provide a more dense reward by aligning the model’s arithmetic outputs with components of a correct solution. Specifically:

1. **Operand Multiset Reward** – rewards based on accurate selection of the correct operand set;
2. **Operand Longest Common Prefix (OLCP)** – rewards weighted by prefix matches between predicted and target operand sequences;
3. **Standardized Longest Common Prefix (SLCP)** – rewards based on longest common prefix matches after canonicalizing arithmetic expressions.

Due to computational inefficiencies observed with SLCP (e.g., recursive expression standardization during rollout), our experiments focus primarily on Operand Multiset and OLCP. We aim to understand whether these denser reward signals can improve the learning efficiency and arithmetic reliability of LLMs under the RLOO training regime.

In this work, we find that partial-credit rewards, especially OLCP, improve training stability and reward density without sacrificing correctness. OLCP achieves higher mean rewards and format accuracy than both the baseline RLOO and multiset variants, highlighting the value of structured prefix-based feedback. Our results demonstrate that fine-grained reward shaping can enhance arithmetic reasoning in small LLMs, and offer a step toward more robust symbolic RL training.

2 Related Work

Language models, even at large scales, continue to struggle with reliable arithmetic and symbolic reasoning. Benchmark tasks such as GSM8K (Cobbe et al., 2021) and MATH (Hendrycks et al., 2021) have revealed persistent brittleness in multi-step numerical reasoning, where models often rely on surface-level heuristics rather than executing precise computation. These challenges are amplified in tasks like COUNTDOWN, where a single misstep invalidates an entire solution, underscoring the limitations of current token-level training paradigms.

Recent efforts to improve mathematical reasoning have focused on math-specific pretraining and instruction fine-tuning. DeepSeekMath (DeepSeek-AI et al., 2025) introduces a math-centric corpus to improve symbolic fluency, while curated instruction datasets have shown gains on GSM8K and MATH through supervised tuning (Li et al., 2025). Despite these improvements, such methods still operate under token-only training objectives and rely on sparse pass/fail evaluation. They offer no built-in mechanism for modeling intermediate computations or for reinforcing partial progress, limiting their ability to guide models toward precise arithmetic execution.

Reinforcement learning has been explored as an alternative training paradigm (Li et al., 2024; Hu et al., 2025; Zhang et al., 2025), particularly in tasks like COUNTDOWN, where symbolic correctness and exploration are critical. For example, Pan et al. (2025) and the CS224R baseline use PPO-style training with a verifier that awards a reward of 1 only for exact target matches. While these methods achieve moderate accuracy (0.12–0.28), learning often stagnates: every incorrect expression yields zero reward, resulting in collapsed exploration and unstable training dynamics.

Outside of language modeling, dense rewards have been widely used to improve sample efficiency in structured reinforcement learning settings. Hindsight Experience Replay (HER) (Andrychowicz et al., 2018) enables agents to learn from failed trajectories by relabeling goals based on partial success, while shaped rewards have helped RL agents learn long-horizon tasks with sparse end-goals (Vasan et al., 2024; Memarian et al., 2021). These approaches demonstrate the power of structured intermediate feedback in brittle problem spaces—an insight we aim to bring into the context of language model fine-tuning for arithmetic.

Closer to our domain, prefix-matching techniques have been proposed to assign structured partial rewards in symbolic tasks. (Zhu et al., 2025; Mudgal et al., 2024; Lee and Tong, 2025) apply prefix-based rewards to logic reasoning, demonstrating that intermediate correctness can significantly stabilize learning. Our work extends this idea to arithmetic reasoning, designing operand- and expression-level reward signals that align with the symbolic structure of arithmetic outputs and can be feasibly computed during training.

Finally, this work builds on a broader line of research into reward optimization for language models. Reinforcement learning from human feedback (RLHF) has shown that carefully designed reward signals can shape model behavior to align more closely with human preferences (Ouyang et al., 2022). We adopt a similar philosophy in the context of arithmetic reasoning—not to enforce subjective

alignment, but to construct structured, interpretable signals that guide the model toward precise, stepwise numerical computation.

3 Method

3.1 Supervised Fine-Tuning (SFT)

We followed the Supervised Fine-Tuning (SFT) procedure as outlined in the project specification. The base model, Qwen 2.5 0.5B, was fine-tuned using the WarmStart dataset through standard next-token prediction. The loss was applied only to the completion tokens, enabling the model to imitate high-quality rollouts from the dataset. This step served as a warm-start initialization before reinforcement learning.

3.2 Online Reinforcement Learning with Leave-One-Out Baseline (RLOO)

To improve beyond imitation, we implemented online policy-gradient fine-tuning using the RLOO algorithm. This method updates the policy based on the relative advantage of each sampled rollout compared to the average reward of the others, which reduces gradient variance and promotes learning from higher-reward trajectories. We optimize the following objective:

$$\frac{1}{k} \sum_{i=1}^k \left[R(y_{(i)}, x) - \frac{1}{k-1} \sum_{j \neq i} R(y_{(j)}, x) \right] \nabla \log \pi(y_{(i)} | x) \quad \text{for } y_{(1)}, \dots, y_{(k)} \stackrel{\text{i.i.d.}}{\sim} \pi_{\theta}(\cdot | x)$$

We experimented with a range of hyperparameters:

- **Learning Rate:** We tested learning rates from 2×10^{-8} to 2×10^{-5} , ultimately choosing 1×10^{-6} for its stability and consistent performance.
- **Temperature:** We settled on 0.3 initially, then increased to 0.5 during checkpoint stacking to encourage exploration on difficult prompts.
- **Generation Length:** We increased the generation length from 512 to 768 tokens to reduce premature truncation and better capture multi-step reasoning.
- **Rollouts (k):** We found that higher values of k improved exploration. $k = 12$ to $k = 16$ produced the most diverse and informative samples for gradient updates.
- **Gradient Clipping:** We used a max norm of 1.0; reducing it to 0.5 yielded no clear benefit.
- **KL Divergence:** Adding a KL penalty had negligible effect on our training stability and performance, so we omitted it for simplicity.
- **Batch Size:** Smaller batches (size 1) were more stable in training; larger sizes (2–4) led to higher variance in gradients.

Our final configuration for the reinforcement learning model used a long generation length (768), high rollout count ($k = 16$), small learning rate, and temperature scaling to improve exploration. Despite this, many rollouts continued to receive zero reward due to the sparsity of the rule-based signal, motivating the need for more informative reward shaping.

3.3 Extension: Partial Credit for Reward Shaping

To address the challenge of sparse and binary rewards in the COUNTDOWN task, we introduced partial credit signals during RLOO using a pre-computed oracle database of valid solutions. We used the `countdown_numbers_solver` to pre-compute an oracle database of valid solutions for the Countdown-Tasks-3to4 dataset (Xie, 2020). This allowed us to evaluate and assign intermediate rewards to approximate-but-incorrect outputs.

We implemented three reward shaping variants:

1. **Operand Multiset:** This reward assigns partial credit based on the accuracy of the operands used, regardless of their order or the operations applied. We calculate the operand multisets of the response’s solution and the oracle solutions, calculate the highest accuracy of the response’s multiset with respect to the various solutions, and scale the highest match as the reward. It encourages models to select correct inputs even when the full computation is incorrect. More formally, let R be the response operand multiset, β be the scale factor, and $\mathcal{O} = \{O_1, \dots, O_n\}$ the set of oracle operand multisets. Then:

$$\text{MultisetReward}(R) = \beta \times \max_{O_i \in \mathcal{O}} \frac{|R \cap O_i|}{|O_i|}.$$

2. **Operand Longest Common Prefix (OLCP):** This variant rewards outputs proportionally to the length of the longest common prefix (LCP) between the model’s operand sequence and the oracle solutions’. For the final reward, we take the length of the LCP, divide it by the length of the response’s solution, and scale it such that it never exceeds a perfect score of 1.0. This provides a finer reward than the operand multiset reward by accounting for the order of operands, for partially correct reasoning steps. Let r be the operand sequence in the response, β be the scale factor, and $\mathcal{O}$ the set of oracle operand sequences. Then:

$$\text{OLCPReward}(r) = \beta \times \max_{o_i \in \mathcal{O}} \frac{\text{LCP}(r, o_i)}{|r|}.$$

3. **Standardized Longest Common Prefix (SLCP):** This prefix reward is the ‘strictest’ grader, and rewards responses proportionally to the longest common prefix of numbers and operands used in the solution. During some preliminary qualitative analyses of LLM responses, we noticed the use of excessive parentheses (e.g.: $((1+2)+3))$), or slight variations in how it represents certain operations. To account for this, we precomputed the standardized version of the oracle database, and normalize each response before calculating the LCP reward. During standardization, we map operator symbols to a common set (e.g., converting “x” to “*”) and remove redundant parentheses. For the final reward, we take the length of the LCP, divide it by the length of the response’s solution, and scale it such that it never exceeds a perfect score of 1.0. This rewards structural correctness and formatting alignment in addition to operand choice. Let $\text{Std}(x)$ be the standardized form of expression x . Then:

$$\text{SLCPReward}(r) = \max_{o_i \in \mathcal{O}} \frac{\text{LCP}(\text{Std}(r), \text{Std}(o_i))}{|\text{Std}(r)|}.$$

For all partial credit rewards, we retain some logic from Gandhi et al.: a fully correct solution receives a score of 1.0, and a response without proper `<answer>` tags receives a 0.0 (Gandhi et al., 2025). We also retain the concept of a ‘format score’: if the response uses proper `<answer>` tags but do not include operands to extract, we give it a format bonus of 0.1. Otherwise, we proceed with a denser reward function based on the operand multiset, OLCP, or SLCP partial credit reward system.

All partial credit rewards are designed such that the final reward never exceeds 1.0, which is only rewarded to solutions that evaluate to the target. By introducing these partial credit mechanisms, we aimed to make the reward function denser and more informative, ultimately enabling the model to learn from imperfect but promising rollouts.

Using the most promising hyperparameters from section 3.2, we continue to experiment with hyperparameters with the partial credit reward mechanisms. For all experiments, we used RLOO to use reinforcement learning on the WarmStart SFT model from the milestone. Primarily, we focus on tuning:

- **Scaling Factor:** Both OLCP and operand multiset use a scaling factor β to compute the final reward. For the operand multiset reward mechanism, we experiment with a scaling factor of 0.5 and 0.7; for the OLCP mechanism, we experiment with 0.7 and 0.9.
- **Generation Length and Rollouts (k):** During RLOO training, we observed that higher values of k improved exploration and training stability. However, responses were still being cut off before it could arrive at a final solution, so we continued to experiment with generation length. Due to compute memory limitations, we experimented with how to balance generation length and rollout number, testing generation lengths of 512, 768, and 1028, and $k = 8, 9, 12$.

We evaluate our models using the last 1000 examples of the Countdown-Tasks-3to4, and calculate its mean score, a ‘strict accuracy’ and a ‘format accuracy’: response scores of 1 increase the strict accuracy, while scores ≥ 0.1 increase the format accuracy.

4 Results and Discussion

4.1 Reward and Hyperparameter Experimentation

After preliminary experimentation with the three partial credit reward mechanisms, we ultimately decided to eliminate the SLCP mechanism from consideration. While training the RLOO model, we noticed the model’s tendency to use unnecessary parentheses. Thus, our implementation of the SLCP reward requires a step to first standardize the solution generated by the LLM by removing redundant parentheses and standardizing the operators used. This was our attempt at allowing equivalent expressions to earn partial credit points as well: for example, given an oracle solution of $1+2-3$, an LLM solution of $(1+3)-2$ would have a strict LCP of 0 (since ‘1’ $\neq$ ‘(’); after standardization, it should receive an LCP of 2, for matching the prefix ‘1+’. Unfortunately, this requires a recursive algorithm, which proved to add too much overhead while generating rollouts for training.

The results of our hyperparameter tuning for the operator multiset and OLCF reward mechanisms can be seen in Table 1, including the evaluation results of our RL model which used the original COUNTDOWN reward. The highest mean reward score (0.3631) was achieved using the OLCF reward with a generation length of 768 and reward scaling of 0.9. This configuration also produced the highest format accuracy (85.90%) and one of the highest strict accuracies (30.80%). In comparison, the original reward setup yielded a mean reward of 0.3263, with 29.30% strict accuracy and 62.60%format accuracy. All multiset reward configurations resulted in lower performance across metrics, with mean rewards ranging from 0.1825 to 0.2301 and strict accuracies between 16.50% and 19.40%.

Table 1: Evaluation Results for hyperparameter experimentation for partial credit rewards.

Method	Reward Scaling	Gen Length	k	Mean Reward	Strict Acc.	Format Acc.
RLOO	n/a	512	16	0.3263	29.30%	62.60%
RLOO w/ Multiset	0.5	1024	8	0.2301	19.40%	59.90%
		768	12	0.1825	16.70%	55.30%
	0.7	1028	9	0.2097	16.50%	57.80%
		768	12	0.2252	19.00%	54.20%
RLOO w/ OLCF	0.7	1024	8	0.3555	31.40%	72.90%
		768	12	0.3400	29.70%	72.70%
	0.9	1024	9	0.3285	27.90%	77.40%
		768	12	0.3631	30.80%	85.90%

While the operand multiset reward was designed to provide denser supervision by granting partial credit for correctly selecting operations, our experiments showed that it consistently failed to lead to improvements over the original COUNTDOWN reward system. During training, the graphs for mean training reward looked relatively stable with comparable rewards to the OLCF training graphs - regardless, it underperformed all other methods in evaluation. A potential explanation is that, although the multiset reward increases the frequency of non-zero feedback for a ‘denser’ reward signal, it may be too dense without being discriminative in a helpful way, leading to reward hacking. The operator multiset mechanism assigns points based on whether or not the model’s response uses the same operands with the oracle solutions, while ignoring operand order and how accurately each operand is used. Additionally, given the limited set of operations in COUNTDOWN (‘+’, ‘-’, ‘*’, ‘/’), the model can often receive a non-trivial reward (e.g., >0.1) simply by assembling plausible-looking expressions with correct or nearly correct operands. Thus, it is possible that the model learned to exploit this reward mechanism by generating solutions with arbitrary operations and earning rewards during training without improving its underlying reasoning. Therefore, the operand multiset mechanism seemed to provide limited guidance for learning structured computation. So although the reward

signal provided is more frequent, it lacks the granularity and pressure needed to drive meaningful improvements in arithmetic accuracy or solution structure.

Using the OLCB reward mechanism with RLOO granted better results, achieving a strict accuracy that remained on par with the original Countdown reward ($\approx 30\%$), while both mean reward and format accuracy improved. Unlike the operand multiset reward, which only checks whether the correct operands appear somewhere in the expression, OLCB also requires that they occur in the same order as one of the oracle solutions, making it harder for the policy to hack the reward by arbitrarily using the operands in an otherwise incorrect solution. Consequently, OLCB supplies a more discriminative signal compared to the multiset reward, and a denser signal compared to the original reward, guiding the agent toward better-structured outputs.

Notably, most of the mean-reward gain seems to originate from the jump in format accuracy rather than from an improved logical correctness and strict accuracy. There are two potential explanations for this. First off, the longer generation length used during training (1024/768 tokens vs. 512 for the baseline) could have provided the model with more room to generate a response with well-formed `<answer>` tags and final answers, providing more non-zero rewards during training, and generally improving the general structure of the responses generated. Another contributing factor is the reward shaping itself. The original Countdown rewards provides a maximum ‘format bonus’ of 0.1 for a correctly closed `<answer>` block. On the other hand, OLCB rewards responses with denser credit for using operands in the correct order, regardless of the numbers used in the arithmetic expression. This richer signal may encourage the model to “think faster” and produce shorter, fully formatted responses that secure more partial credit early, even when the arithmetic reasoning is not fully correct. Given that the strict accuracies of the OLCB models were comparable with the original RL model, there was only a minor improvement on the models’ underlying arithmetic reasoning. Instead, the models seem to have learned to prioritize scoring the ‘easier’ partial credit points by formatting their answers correctly, rather than ‘thoroughly thinking out’ the arithmetic to produce a fully correct answer within the `<answer>` tags. In effect, OLCB biases learning toward complete, syntactically valid outputs while still nudging the policy toward the correct operand sequence—explaining why format accuracy climbs sharply and mean reward rises without a significant improvement in strict accuracy.

4.2 Reward Collapse & Qualitative Failure Analysis

Across all reward mechanisms, including the original Countdown scoring system, we observe training graphs with irreversible reward collapse. Figure 1 exhibits the training graphs of during our hyperparameter experimentation for the OLCB reward method.

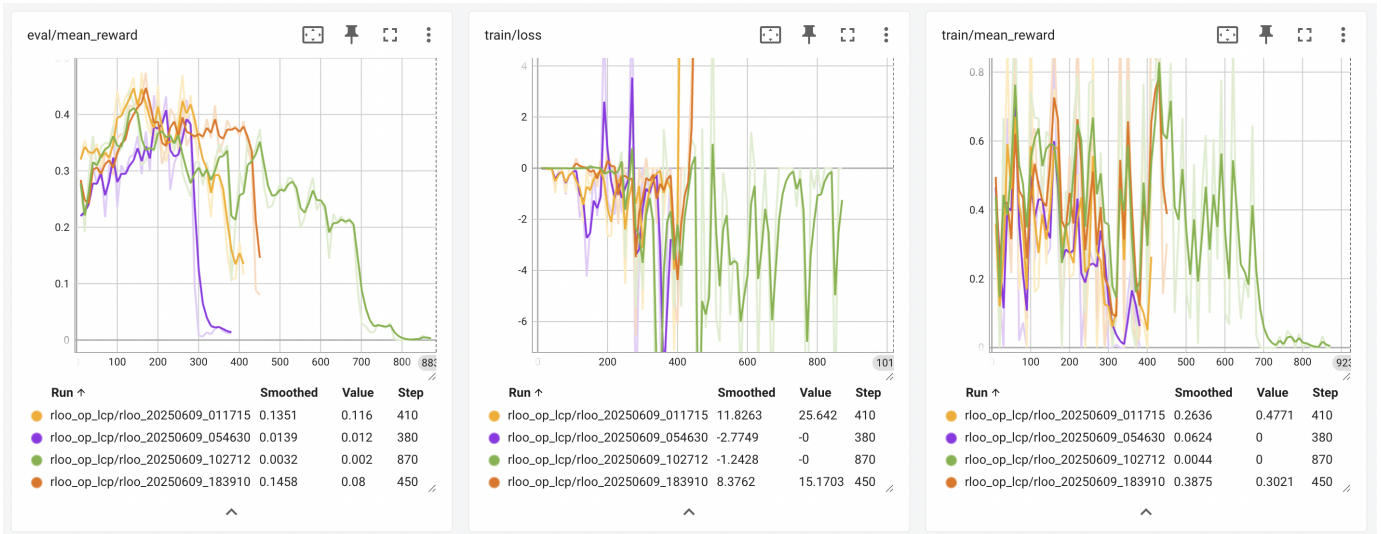


Figure 1: Eventual reward collapse across all hyperparameter combinations while training RLOO with the OLCB reward mechanism.

Across all four hyper-parameter settings, the agent’s evaluation return climbs steadily until ≈ 150 -250 training steps, after which it collapses to near-zero and never recovers. A matching collapse is visible in the mean training return as well, while the training loss spikes and oscillates. At this point, the model seems to have ‘forgotten’ its previously learned representations and policies. Once the network parameters have crossed this boundary, it no longer outputs properly formatted responses, as seen in Failure Mode Example 1 (a response we took note of during training), and subsequent gradients are dominated by zero rewards, which prevents the agent from relearning its earlier, higher-performing behaviour and rendering the crash effectively irreversible within the same training session.

Failure Mode Example 1: Response does not contain any meaningful information and mostly consists of end of text tags

```
Let<|endoftext|><|endoftext|><|endoftext|><|endoftext|>
<|endoftext|><|endoftext|><|endoftext|><|endoftext|>
<|endoftext|><|endoftext|><|endoftext|><|endoftext|>
<|endoftext|><|endoftext|><|endoftext|><|endoftext|>
<|endoftext|><|endoftext|><|endoftext|><|endoftext|>
<|endoftext|><|endoftext|><|endoftext|><|endoftext|>...
```

Before this point of irreversible collapse, we noticed another trend exhibiting the multiset reward hacking. Failure mode example 2 below exhibits one example of this reward hacking, when learned by the operand multiset reward mechanism. Given the numbers [8, 5, 33, 16] and a target of 26, the LLM responded with a solution that disregards the numbers used, leaving out half of the numbers required, and repeating one of the numbers used. Interestingly, one of the oracle solutions happened to be $33 - (16/8 + 5)$, of which 2/3 of the operands are the same, and would achieve a relatively high multiset reward.

Failure Mode Example 2: Response uses operands correctly but has incorrect arithmetic

Numbers: 8, 5, 33, 16, Target: 26

Let me try to find a way to get to 26 using these numbers.
First, let me try working backwards from 26.
26 is an even number, so it could be helpful to get an even number first.

Let me try some operations:

1. 16 is closest to 26, so let’s try to get from 16 to 26
2. We need to add 10 to 16 to get 26
3. Can we get 10 using 8 and 33?
4. 33 is too large, but maybe we can use division
5. Let’s try: $33/33 = 1$
6. So if we get 1, add it to 16, we’ll reach 26

I can get 1 by dividing 33 by 33!
Then we can add this 1 to 16 to get 26.

Let me verify:
 $33/33 = 1$
 $1 + 16 = 26$
Perfect! We’ve found our solution.
</think>
<answer> $(33/33 + 16)$ </answer>

Numbers: [8, 5, 33, 16], Target: 26
Score: 0.1

5 Limitations and Future Work

While our use of dense partial-credit rewards improved training stability and sample efficiency, several limitations remain. First, we only explored a narrow slice of the hyperparameter space due to computational constraints. More extensive tuning—especially around rollout count, reward scaling schedules, and exploration temperature—could yield further gains in both accuracy and stability.

Second, our approach is limited to reward shaping over textual outputs. It evaluates only the final form of the generated arithmetic expression, rather than the reasoning steps used to derive it. Future work could investigate fine-grained trajectory-level feedback—e.g., rewarding correct intermediate computations or decomposed plans—potentially through automatic reasoning trace extraction or multi-step verifier tools.

Third, our experiments focus purely on in-model computation. Integrating external symbolic tools (e.g., a calculator or program executor) during training could help bridge gaps between surface-level fluency and actual numerical reasoning. Coupled with tool-use-aware reward design, this might enable the model to develop better arithmetic competence through grounding.

Finally, although OLCP improved partial credit learning, all variants eventually suffered from reward collapse. Future work should explore more robust RL algorithms (e.g., variance-controlled baselines, entropy bonuses, or trajectory relabeling) and curriculum-based strategies to sustain exploration and avoid degenerate convergence.

6 Conclusion

Our experiments demonstrate that denser, structure-aware rewards can partially mitigate the brittleness of arithmetic fine-tuning for small LLMs. Among the mechanisms we explored, the *Operand Longest Common Prefix* (OLCP) reward struck the best balance between density and discriminativeness: it preserved strict accuracy while boosting overall reward and format correctness, suggesting that prefix-level guidance helps policies converge on syntactically valid, better-structured outputs. In contrast, the *Operand Multiset* reward—though frequent—proved too coarse, encouraging superficial operand reuse without genuine reasoning gains, and thus under-performed the original COUNTDOWN signal. Yet even with OLCP, training trajectories eventually collapsed, revealing a persistent vulnerability to reward hacking and mode collapse in RLOO. Taken together, these findings underline two takeaways: (i) thoughtfully crafted partial-credit signals can make arithmetic RLHF notably more sample-efficient, but (ii) stabilizing long-horizon symbolic RL will likely require complementary advances such as adaptive scaling, variance-reduction baselines, or hybrid search-and-execute curricula. We hope this work provides a stepping-stone toward more robust, numerate language models and spurs further investigation into principled reward shaping for symbolic reasoning tasks.

7 Team Contributions

Since the milestone, our team members have worked on:

- **Alice Chen** Implemented partial credit rewards; experimented with OLCP hyperparameters
- **Ryan Lian** Implemented partial credit rewards; experimented with operator multiset hyperparameters
- **Luke Tchang** Implemented RLOO algorithm and dataloaders; experimented with RLOO hyperparameters

Changes from Proposal Our project has undergone many revisions since the proposal. Initially, we had hoped to combine experimentation with tool use and a denser reward representation. However, our attempts to identify good hyperparameters for the RLOO model quickly burned through our credits, and we needed to narrow down our focus to prefix-based partial credit reward mechanisms. As such, our team member contributions have changed substantially to reflect these updates.

References

- Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. 2024. Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs. arXiv:2402.14740 [cs.LG] <https://arxiv.org/abs/2402.14740>
- Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. 2018. Hindsight Experience Replay. arXiv:1707.01495 [cs.LG] <https://arxiv.org/abs/1707.01495>
- Tanja Baeumel, Josef van Genabith, and Simon Ostermann. 2025. The Lookahead Limitation: Why Multi-Operand Addition is Hard for LLMs. arXiv:2502.19981 [cs.CL] <https://arxiv.org/abs/2502.19981>
- Johan Boye and Birger Moell. 2025. Large Language Models and Mathematical Reasoning Failures. arXiv:2502.11574 [cs.AI] <https://arxiv.org/abs/2502.11574>
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168 [cs.LG] <https://arxiv.org/abs/2110.14168>
- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojuan Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanbiao Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. 2025. DeepSeek. arXiv:2501.12948 [cs.CL] <https://arxiv.org/abs/2501.12948>
- Kanishk Gandhi, Ayush Chakravarthy, Anikait Singh, Nathan Lile, and Noah D. Goodman. 2025. Cognitive behaviors that enable self-improving reasoners, or, four habits of highly effective stars. <https://arxiv.org/abs/2503.01307>. arXiv preprint.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. arXiv:2103.03874 [cs.LG] <https://arxiv.org/abs/2103.03874>
- Jian Hu, Jason Klein Liu, and Wei Shen. 2025. REINFORCE++: An Efficient RLHF Algorithm with Robustness to Both Prompt and Reward Models. arXiv:2501.03262 [cs.CL] <https://arxiv.org/abs/2501.03262>

- Alan Lee and Harry Tong. 2025. Token-Efficient RL for LLM Reasoning. arXiv:2504.20834 [cs.LG] <https://arxiv.org/abs/2504.20834>
- Haoyang Li, Xuejia Chen, Zhanchao XU, Darian Li, Nicole Hu, Fei Teng, Yiming Li, Luyu Qiu, Chen Jason Zhang, Qing Li, and Lei Chen. 2025. Exposing Numeracy Gaps: A Benchmark to Evaluate Fundamental Numerical Abilities in Large Language Models. arXiv:2502.11075 [cs.CL] <https://arxiv.org/abs/2502.11075>
- Ziniu Li, Tian Xu, Yushun Zhang, Zhihang Lin, Yang Yu, Ruoyu Sun, and Zhi-Quan Luo. 2024. ReMax: A Simple, Effective, and Efficient Reinforcement Learning Method for Aligning Large Language Models. arXiv:2310.10505 [cs.LG] <https://arxiv.org/abs/2310.10505>
- Farzan Memarian, Wonjoon Goo, Rudolf Lioutikov, Scott Niekum, and Ufuk Topcu. 2021. Self-Supervised Online Reward Shaping in Sparse-Reward Environments. arXiv:2103.04529 [cs.LG] <https://arxiv.org/abs/2103.04529>
- Sidharth Mudgal, Jong Lee, Harish Ganapathy, YaGuang Li, Tao Wang, Yanping Huang, Zhifeng Chen, Heng-Tze Cheng, Michael Collins, Trevor Strohman, Jilin Chen, Alex Beutel, and Ahmad Beirami. 2024. Controlled Decoding from Language Models. arXiv:2310.17022 [cs.LG] <https://arxiv.org/abs/2310.17022>
- OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O’Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Ceron Uribe, Andrea Vallone, Arun Vijayvergiya,

- Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] <https://arxiv.org/abs/2303.08774>
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. arXiv:2203.02155 [cs.CL] <https://arxiv.org/abs/2203.02155>
- Jiayi Pan, Junjie Zhang, Xingyao Wang, Lifan Yuan, Hao Peng, and Alane Suhr. 2025. TinyZero. <https://github.com/Jiayi-Pan/TinyZero>. GitHub repository, accessed 2025-01-24.
- Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2025. Qwen2.5 Technical Report. arXiv:2412.15115 [cs.CL] <https://arxiv.org/abs/2412.15115>
- Gautham Vasan, Yan Wang, Fahim Shahriar, James Bergstra, Martin Jagersand, and A. Rupam Mahmood. 2024. Revisiting Sparse Rewards for Goal-Reaching Reinforcement Learning. arXiv:2407.00324 [cs.RO] <https://arxiv.org/abs/2407.00324>
- Tao Xie. 2020. countdown-numbers-solver. <https://github.com/exitexit/countdown-numbers-solver>. GitHub repository, last modified December 9, 2020.
- Yang Yan, Yu Lu, Renjun Xu, and Zhenzhong Lan. 2025. Do PhD-level LLMs Truly Grasp Elementary Addition? Probing Rule Learning vs. Memorization in Large Language Models. arXiv:2504.05262 [cs.CL] <https://arxiv.org/abs/2504.05262>
- Xiaoying Zhang, Hao Sun, Yipeng Zhang, Kaituo Feng, Chaochao Lu, Chao Yang, and Helen Meng. 2025. Critique-GRPO: Advancing LLM Reasoning with Natural Language and Numerical Feedback. arXiv:2506.03106 [cs.CL] <https://arxiv.org/abs/2506.03106>
- Jiace Zhu, Yingtao Shen, Jie Zhao, and An Zou. 2025. Path-Consistency: Prefix Enhancement for Efficient Inference in LLM. arXiv:2409.01281 [cs.CL] <https://arxiv.org/abs/2409.01281>